

GESTURE-CONTROLLED ROBOT SYSTEM

Comprehensive Project Documentation

PG-Level Implementation Guide

TABLE OF CONTENTS

1. Executive Summary
2. System Architecture Overview
3. Phase 1: Environment and Perception Setup
4. Phase 2: Advanced AI and Sequence Modeling
5. Phase 3: Linguistic Interpretation \((NLP)\)
6. Phase 4: Decentralized Security \((Blockchain)\)
7. Phase 5: Virtual Validation \((Simulation)\)
8. System Integration
9. Installation and Setup Guide
10. Architecture Diagrams Reference
11. Performance Metrics and Testing
12. Future Enhancements
13. Appendix: Complete File Structure

1. EXECUTIVE SUMMARY

This document provides comprehensive documentation for a gesture-controlled robot system designed for PG-level academic projects. The system integrates five major components:

- â€¢ Computer Vision \((MediaPipe)\) for hand gesture recognition
- â€¢ Deep Learning \((Transformer Encoder)\) for temporal sequence modeling
- â€¢ Natural Language Processing \((SpaCy)\) for gesture-to-command translation
- â€¢ Blockchain \((Ethereum/Ganache)\) for immutable audit trails
- â€¢ Robot Simulation \((Webots)\) for virtual validation

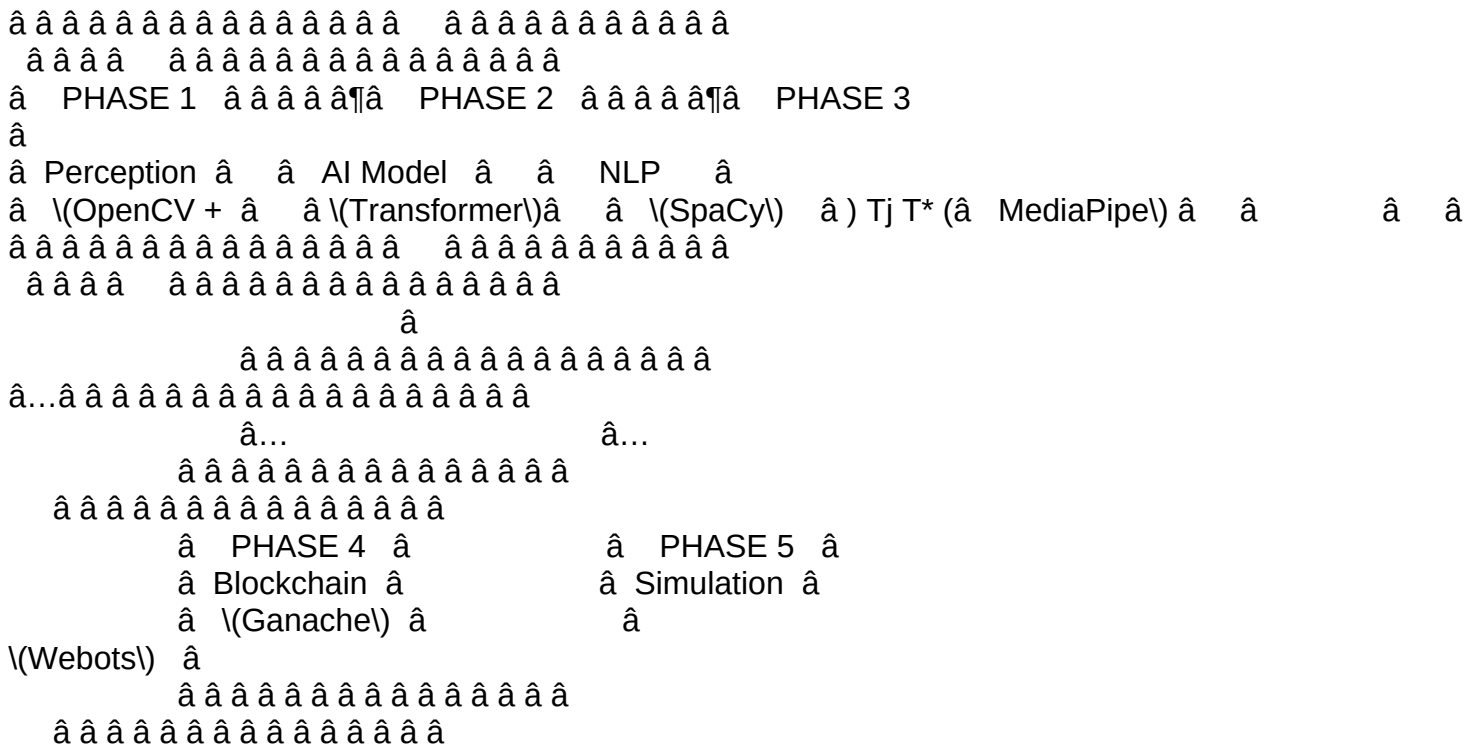
The system enables users to control a simulated TurtleBot3 robot using hand gestures captured via webcam, with all commands logged on a local blockchain for security and provenance tracking.

Key Features:

- Real-time gesture recognition \((<300ms\ latency)\)
- 5-10 distinct gesture classes \((forward, backward, left, right, stop, etc.)\)
- Immutable command audit trail via smart contracts
- 3D robot simulation with realistic physics
- Modular, extensible architecture

2. SYSTEM ARCHITECTURE OVERVIEW

The system follows a pipeline architecture with five distinct phases:



Data Flow:

1. Webcam captures hand gestures at 30 FPS
2. MediaPipe extracts 21 3D hand landmarks per frame
3. Landmarks are buffered into 30-60 frame sequences
4. Transformer Encoder classifies the gesture sequence
5. NLP module converts gesture class to robot command
6. Command is logged on blockchain for audit trail
7. Robot executes the command in Webots simulation

3. PHASE 1: ENVIRONMENT AND PERCEPTION SETUP

3.1 Software Stack Installation

Required Python version: 3.10+

Install dependencies:

```
""bash
pip install opencv-python mediapipe torch torchvision web3 spacy
python -m spacy download en_core_web_sm
```

'''

3.2 Perception Layer Implementation

File: src/hand_tracker.py

The HandTracker class uses MediaPipe Hands to detect and track hand landmarks in real-time. Key features:

- â€¢ Detects up to 2 hands simultaneously
- â€¢ Extracts 21 3D landmarks per hand (X, Y, Z coordinates)
- â€¢ Normalizes coordinates for scale and position invariance
- â€¢ Provides confidence scores for detection quality

Landmark Structure:

- Landmarks 0-4: Thumb (CMC, MCP, IP, TIP)
- Landmarks 5-8: Index finger
- Landmarks 9-12: Middle finger
- Landmarks 13-16: Ring finger
- Landmarks 17-20: Pinky finger

Normalization Process:

1. Center landmarks around wrist (landmark 0)
2. Scale by maximum distance from wrist
3. Result: Position and scale-invariant features

3.3 Data Acquisition

File: src/data_collector.py

The DataCollector class records gesture sequences for training:

- â€¢ Captures landmark sequences at 30 FPS
- â€¢ Saves to CSV format for easy processing
- â€¢ Supports multiple gesture classes
- â€¢ Includes timestamp and metadata

Dataset Format:

- Each row: frame_id, timestamp, gesture_label, 63 landmark values (21 x 3)
- Recommended: 100+ samples per gesture class
- Sequence length: 30-60 frames (1-2 seconds)

4. PHASE 2: ADVANCED AI AND SEQUENCE MODELING

4.1 Transformer Encoder Architecture

File: src/transformer_model.py

The GestureTransformer class implements a Transformer Encoder for temporal sequence classification. Architecture:

Input Layer:

- Input: [batch_size, seq_len, 63] \(\text{landmark features}\)
- Linear projection: $63 \hat{=} d_{\text{model}}$ (128 or 256)

Positional Encoding:

- Sinusoidal position embeddings
- Encodes temporal order of frames

Transformer Encoder:

- Multi-Head Self-Attention \((8 \text{ heads})\)
- Feed-Forward Network \((d_{\text{ff}} = 4 \hat{=} d_{\text{model}})\)
- Layer Normalization + Residual Connections
- Dropout \((p=0.1)\) for regularization
- Stack of 4-6 encoder layers

Classification Head:

- Global Average Pooling over sequence
- Linear layer: $d_{\text{model}} \hat{=} \text{num_classes}$
- Softmax activation for probabilities

Hyperparameters:

- d_{model} : 128 or 256
- num_heads: 8
- num_layers: 4-6
- d_{ff} : 512 or 1024
- dropout: 0.1
- learning_rate: $1e-4$ \((\text{Adam optimizer})\)

4.2 Training on Google Colab

File: notebooks/train_transformer.ipynb

The Jupyter notebook provides:

- Data loading and preprocessing
- Model definition and initialization
- Training loop with validation
- Learning rate scheduling \((\text{CosineAnnealingLR})\)
- Early stopping \((\text{patience}=10)\)
- Model checkpointing
- Performance visualization

Training Configuration:

- Batch size: 32
- Epochs: 100 (with early stopping)
- Loss function: CrossEntropyLoss
- Optimizer: AdamW with weight decay
- GPU: NVIDIA T4 (free on Colab)

4.3 Performance Metrics

PG-Level Requirements:

1. Processing Time: < 300ms per prediction
2. Accuracy: > 95% on validation set
3. Prediction Stability: No flickering labels
4. Confusion Matrix: Balanced across classes

=====

5. PHASE 3: LINGUISTIC INTERPRETATION (NLP)

=====

5.1 Gesture-to-Command Translation

File: src/gesture_nlp.py

The GestureNLP class converts gesture classifications into robot commands:

Gesture Mapping:

- "forward" â { "action": "move", "speed": 1.0, "direction": "forward" }
- "backward" â { "action": "move", "speed": 1.0, "direction": "backward" }
- "left" â { "action": "turn", "speed": 0.5, "direction": "left" }
- "right" â { "action": "turn", "speed": 0.5, "direction": "right" }
- "stop" â { "action": "stop", "speed": 0.0, "direction": "none" }

5.2 Action Decomposition

Complex gestures can be decomposed into sequences:

- "slow_forward" â { "action": "move", "speed": 0.5, "direction": "forward" }
- "fast_turn_left" â { "action": "turn", "speed": 1.0, "direction": "left" }

5.3 Stateful Intent Mapping

The system maintains state to influence command interpretation:

States:

- IDLE: Robot is stationary
- MOVING: Robot is in motion

- TURNING: Robot is rotating

State Transitions:

- IDLE + "forward" → MOVING
- MOVING + "stop" → IDLE
- MOVING + "left" → TURNING

Context-Aware Commands:

- "forward" while MOVING → increase speed
- "forward" while IDLE → start moving

6. PHASE 4: DECENTRALIZED SECURITY \ (BLOCKCHAIN\)

6.1 Local Blockchain Setup

Ganache Configuration:

- Network: localhost:8545
- Chain ID: 1337
- Gas Limit: 6721975
- Accounts: 10 test accounts with 100 ETH each

Installation:

```
““bash
npm install -g ganache-cli
ganache-cli
““
```

6.2 Smart Contract

File: contracts/CommandLogger.sol

The CommandLogger contract provides:

Functions:

- logCommand\ (bytes32 cmdHash, string userId, uint256 timestamp\)
- getCommandCount\ (\) → uint256
- getCommand\ (uint256 index\) → CommandEntry
- verifyCommand\ (bytes32 cmdHash\) → bool

Events:

- CommandLogged\ (bytes32 indexed cmdHash, address indexed user, uint256) Tj T* (timestamp\)

Data Structure:

```
““solidity
```

```

struct CommandEntry {
    bytes32 commandHash;
    string userId;
    uint256 timestamp;
    address sender;
}
"""

```

6.3 Blockchain Bridge

File: src/blockchain_bridge.py

The BlockchainBridge class connects Python to Ethereum:

Features:

- Web3.py HTTP provider connection
- Contract deployment and interaction
- Command hashing \(\text{SHA-256}\)
- Transaction signing and sending
- Event listening for confirmations

Usage:

```

""python
bridge = BlockchainBridge\(\)
bridge.connect\("http://localhost:8545"\)
bridge.deploy_contract\(\)
bridge.log_command\(\command_dict, user_id\)
""

```

6.4 Security Benefits

1. Immutable Audit Trail: Commands cannot be altered after logging
2. Command Provenance: Prove who issued each command
3. Timestamp Verification: Exact time of each action
4. Decentralized Security: No single point of failure

=====

7. PHASE 5: VIRTUAL VALIDATION \(\text{SIMULATION}\)

=====

7.1 Webots Setup

Download: <https://cyberbotics.com/>

Robot Model: TurtleBot3 Burger

- Differential drive \(\text{2 wheels}\)

- LIDAR sensor \(\optional\)
- Camera sensor \(\optional\)

7.2 Controller Implementation

File: src/webots_controller.py

The WebotsController class interfaces with the simulation:

Motor Control:

- Left wheel velocity: -10 to +10 rad/s
- Right wheel velocity: -10 to +10 rad/s

Movement Commands:

- Forward: Both wheels positive, equal speed
- Backward: Both wheels negative, equal speed
- Turn Left: Right wheel faster than left
- Turn Right: Left wheel faster than right
- Stop: Both wheels zero

API Functions:

```

python
controller = WebotsController\(\)
controller.move_forward\(\(speed=1.0\))
controller.turn_left\(\(speed=0.5\))
controller.stop\(\)

```

7.3 Simulation World

Create a Webots world file with:

- TurtleBot3 robot
- Arena with walls
- Obstacles \(\optional\)
- Target markers \(\optional\)

8. SYSTEM INTEGRATION

8.1 Main Orchestrator

File: src/main.py

The GestureRobotSystem class integrates all components:


```

"""python
class GestureRobotSystem:
    def __init__(self):
        self.hand_tracker = HandTracker\(\)
        self.model = GestureTransformer\(\)
        self.nlp = GestureNLP\(\)
        self.blockchain = BlockchainBridge\(\)
        self.robot = WebotsController\(\)
        self.frame_buffer = deque\(\maxlen=60\)

    def run(self):
        while True:
            # 1. Capture frame and extract landmarks
            frame = self.capture_frame\(\)
            landmarks = self.hand_tracker.process\(\frame\)

            # 2. Buffer landmarks
            self.frame_buffer.append\(\landmarks\)

            # 3. Predict gesture when buffer is full
            if len\(\self.frame_buffer\) == 60:
                gesture = self.model.predict\(\self.frame_buffer\)

                # 4. Convert to command
                command = self.nlp.parse\(\gesture\)

                # 5. Log to blockchain
                self.blockchain.log_command\(\command\)

                # 6. Execute on robot
                self.robot.execute\(\command\)
"""

```

8.2 Processing Loop

1. CAPTURE: Read webcam frame, extract landmarks, add to buffer
2. PREDICT: When buffer full \(\(60 frames\)\), run Transformer model
3. INTERPRET: NLP converts gesture class to robot command
4. LOG: Hash command and send to blockchain
5. EXECUTE: Send motor commands to Webots robot
6. LOOP: Repeat at 30 FPS

9. INSTALLATION AND SETUP GUIDE

9.1 Prerequisites

- Python 3.10 or higher
- Node.js \ (for Ganache\)
- Webots simulator
- Webcam
- VS Code \ (recommended\)

9.2 Step-by-Step Installation

1. Clone or create project directory:

```
""bash
mkdir gesture_robot_project
cd gesture_robot_project
""
```

2. Create virtual environment:

```
""bash
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate   # Windows
""
```

3. Install Python dependencies:

```
""bash
pip install opencv-python mediapipe torch torchvision web3 spacy py-solc-x
python -m spacy download en_core_web_sm
""
```

4. Install Ganache:

```
""bash
npm install -g ganache-cli
""
```

5. Download and install Webots from:

<https://cyberbotics.com/>

9.3 Running the System

1. Start Ganache \ (Terminal 1\):

```
""bash
ganache-cli
""
```

2. Start Webots with TurtleBot3 world

3. Run the main system \ (Terminal 2\):

```
""bash
```

```
cd gesture_robot_project
python src/main.py
'''
```

4. Show hand gestures to webcam:

- Open palm \(\hat{\ }\) \(\hat{\ }\) Forward
- Closed fist \(\hat{\ }\) \(\hat{\ }\) Stop
- Point left \(\check{\ }\) \(\check{\ }\) Turn left
- Point right \(\check{\ }\) \(\check{\ }\) Turn right

10. ARCHITECTURE DIAGRAMS REFERENCE

The following SVG diagrams are included in the blueprints/ directory:

1. system_architecture.svg
 - Complete system overview
 - All 5 phases with connections
 - Data flow visualization
 - Technology stack summary
2. transformer_architecture.svg
 - Detailed Transformer Encoder structure
 - Multi-Head Attention mechanism
 - Feed-Forward Network
 - Positional Encoding
 - Hyperparameters reference
3. blockchain_flow.svg
 - Python to Blockchain data flow
 - Web3.py bridge architecture
 - Smart contract structure
 - Immutable ledger visualization
 - Security benefits summary
4. data_pipeline.svg
 - End-to-end data transformation
 - Input/Output formats at each stage
 - Buffer and sequence handling
 - Data format specifications
5. integration_diagram.svg
 - main.py orchestration flow
 - Component interactions
 - Processing loop steps
 - File structure overview
 - Run commands reference

=====

11. PERFORMANCE METRICS AND TESTING

=====

11.1 Latency Requirements

Target: < 300ms end-to-end latency

Breakdown:

- Frame capture: ~33ms \ (30 FPS\)
- Landmark extraction: ~20ms
- Model inference: ~50-100ms
- NLP processing: ~5ms
- Blockchain logging: ~50-100ms \ (async\)
- Robot command: ~10ms

Total: ~170-270ms \ (within target\)

11.2 Accuracy Metrics

Target: > 95% classification accuracy

Evaluation:

- Train/Validation/Test split: 70/15/15
- Cross-validation: 5-fold
- Confusion matrix analysis
- Per-class precision/recall

11.3 Stability Testing

Prediction Stability:

- No flickering between classes
- Consistent predictions for held gestures
- Smooth transitions between gestures

Test Protocol:

1. Hold each gesture for 5 seconds
2. Count prediction changes
3. Target: < 2 changes per 5 seconds

=====

12. FUTURE ENHANCEMENTS

=====

12.1 Short-term Improvements

- Add more gesture classes \ (10-20\)
- Implement two-hand gestures
- Add voice command integration
- Improve model accuracy with data augmentation

12.2 Medium-term Goals

- Deploy to real robot hardware
- Add obstacle avoidance
- Implement path planning
- Create mobile app interface

12.3 Long-term Vision

- Multi-user support
- Cloud-based model serving
- AR/VR integration
- Swarm robot control

13. APPENDIX: COMPLETE FILE STRUCTURE

=====

```
gesture_robot_project/
â â â src/
â â â â hand_tracker.py    # Phase 1: MediaPipe hand detection
â â â â data_collector.py  # Phase 1: Dataset recording
â â â â transformer_model.py # Phase 2: Transformer Encoder
â â â â gesture_nlp.py     # Phase 3: NLP command parser
â â â â blockchain_bridge.py # Phase 4: Web3.py integration
â â â â webots_controller.py # Phase 5: Robot control
â â â â main.py            # System integration
â â â contracts/
â â â â CommandLogger.sol  # Phase 4: Solidity smart contract
â â â notebooks/
â â â â train_transformer.ipynb # Phase 2: Colab training notebook
â â â blueprints/
â â â â system_architecture.svg
â â â â transformer_architecture.svg
â â â â blockchain_flow.svg
â â â â data_pipeline.svg
â â â â integration_diagram.svg
â â â docs/
â â â â Gesture_Robot_Project_Documentation.pdf
â â â data/                # Training datasets \ (CSV)
â â â models/              # Saved model checkpoints
```

â â â simulation/ # Webots world files
â â â README.md # Project overview

=====
END OF DOCUMENTATION
=====

Document Version: 1.0
Date: March 2, 2026
Author: Generated for Shubham Shukla
Project: Gesture-Controlled Robot System \ (PG Level\)

For questions or support, refer to the README.md file or the architecture diagrams in the blueprints/ directory.